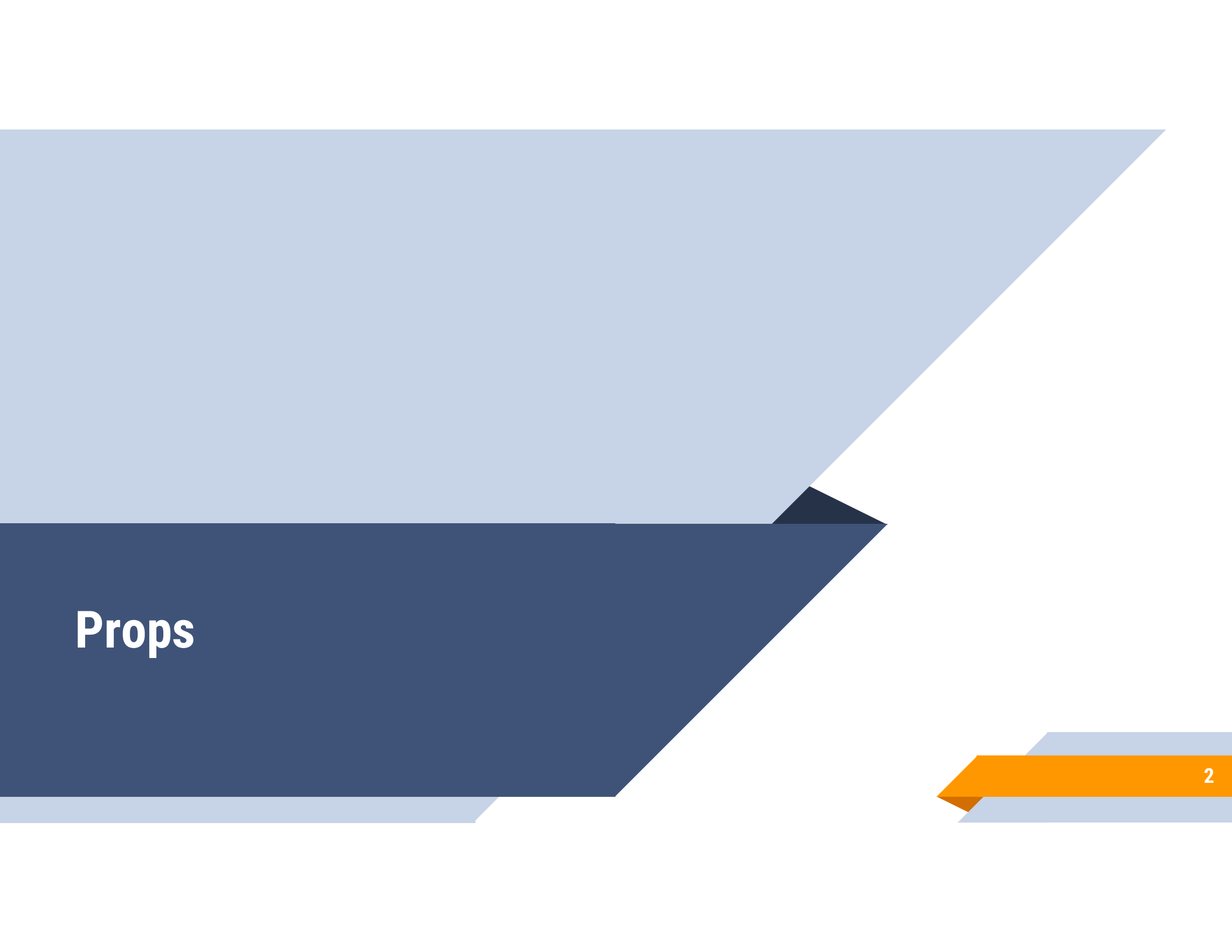


LẬP TRÌNH FRONTEND

The background features a large, dark blue chevron shape pointing to the right, which contains the text. Above and below this shape are lighter blue horizontal bands. At the bottom, there is a solid orange horizontal bar that also has a slight 3D effect.

The background features a large, light blue trapezoidal shape on the left side. Below it is a dark blue trapezoidal shape. On the right side, there is a small orange trapezoidal shape. The word "Props" is written in white on the dark blue shape.

Props

Props

- Dùng để truyền tham số cho component
- Không thể cập nhật từ bên trong component
- Chỉ có thể được tạo lại từ component cha với giá trị mới

Props

```
function Avatar({ person, size }) {  
  return (  
    <img  
      className="avatar"  
      src={getImageUrl(person)}  
      alt={person.name}  
      width={size}  
      height={size}  
    />  
  );  
}
```

```
export default function Profile() {  
  return (  
    <div>  
      <Avatar  
        size={100}  
        person={{  
          name: 'Katsuko Saruhashi',  
          imageId: 'Yfe0qp2'  
        }}  
      />  
      <Avatar  
        size={100}
```

Props

```
function Avatar({ person, size }) {  
  // ...  
}
```

```
function Avatar(props) {  
  let person = props.person;  
  let size = props.size;  
  // ...  
}
```

Props

- Lưu ý phân biệt cách khai báo 1 tham số props và cách dùng cú pháp “destructuring”
- Đặc biệt: props.children chứa nội dung giữa cặp thẻ mở/đóng của component

Props

```
function Card({ children }) {  
  return (  
    <div className="card">  
      {children} ← <Avatar />  
    </div>  
  );  
}
```

<Card>
<Avatar />
</Card>

Props

- Lưu ý khi render từ một danh sách thì mỗi component phải có thuộc tính key để có thể theo dõi chính xác sự thay đổi, tránh rerender không cần thiết
- Key này không cần phải là duy nhất trong cả trang, chỉ cần duy nhất trong danh sách hiện tại
- Không được lấy chỉ số mảng hoặc `Math.random()` làm key

State

State

- Dùng để lưu dữ liệu của component
- Có thể cập nhật từ bên trong component
- Khi cập nhật sẽ rerender component

State

```
import { useState } from "react";  
function App() {  
  const [count, setCount] = useState(0);  
  return (  
    <>  
      <button onClick={() => setCount(count + 1)}>  
        {count}  
      </button>  
    </>  
  );  
}
```

State

```
const [state, setState] = useState(initialState)
```

Dữ liệu cần lưu

Hàm cập nhật dữ liệu

Giá trị khởi tạo
(hoặc hàm trả về giá
trị khởi tạo)

setState()

- Hàm cập nhật sẽ không thay đổi giá trị ngay mà sẽ ghi nhớ để thực hiện trong lần render tiếp theo
- Do đó sau khi gọi `useState()` xong thì giá trị của state vẫn là giá trị cũ

setState()

```
<button  
  onClick={() => {  
    setCount(count + 1);  
    console.log(count);  
  }}  
>  
  {count}  
</button>
```

setState()

- Hàm cập nhật có thể nhận vào giá trị mới hoặc một hàm trả ra giá trị mới

setState()

```
<button
  onClick={() => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    console.log(count);
  }}
>
  {count}
</button>
```



```
<button
  onClick={() => {
    setCount(count => count + 1);
    setCount(count => count + 1);
    setCount(count => count + 1);
    console.log(count);
  }}
>
  {count}
</button>
```

setState()

- Nếu giá trị mới bằng giá trị cũ thì sẽ không rerender
- Do đó KHÔNG thể cập nhật trực tiếp mảng/đối tượng mà phải tạo lại mảng/đối tượng mới

setState()



```
// Cập nhật vị trí i
arr[i] = x;
setArr(arr);
// Thêm phần tử mới vào cuối mảng
arr.push(x);
setArr(arr);
// Thêm phần tử mới vào đầu mảng
arr.unshift(x);
setArr(arr);
// Xóa phần tử tại vị trí i
arr.splice(i, 1);
setArr(arr);
// Xóa phần tử có giá trị x
arr.splice(arr.indexOf(x), 1);
setArr(arr);
```



```
setArr(arr.map((v, k) => (k === i ? x : v)));

setArr([...arr, x]);

setArr([x, ...arr]);

setArr(arr.filter((v, k) => (k !== i)));

setArr(arr.filter((v) => (v !== x)));
```

setState()



```
obj.a = x;  
setObj(obj)
```



```
setObj({...obj, a: x});
```

setState()

- Nên lưu giá trị của các thẻ input vào state và cập nhật trong sự kiện onChange thay vì đặt id và gọi document.getElementById() như code JS thông thường
- onChange của React $\approx$ onChange của JS
- Thẻ có value + không có onChange = readonly

setState()

```
<input
  type="text"
  value={name}
  onChange={(e) => setName(e.target.value)}
  placeholder="Enter your name"
/>
<p>Hello, {name}</p>
```